

/ The design had flaws:*

1- the memory had inverted reset so the memory never write or read data

2- the memory made a wrong AND operation with the data written which lead to corrupted data

3- the slave used to send address and receive data from memory in same cycle that made data off by one cycle

4- the slave used wrong address boundaries where it append beats and if the beats exceeded the 4kb it rejects the rest of the burst. it should reject the whole burst.

**/*

```

4
5   `include "uvm_macros.svh"
6   import uvm_pkg::*;
7   import param_pkg::*;
8
9   class axi_transaction extends uvm_sequence_item;
10
11   //////////////////////////////////////////////////
12   // 1. WRITE SIGNALS
13   //////////////////////////////////////////////////
14   rand logic [ADDR_SIZE-1:0] AWADDR;
15   rand logic [      2:0] AWSIZE;
16   rand logic [      7:0] AWLEN;
17   logic      AWVALID;
18   logic      AWREADY;
19
20   rand logic [DATA_SIZE-1:0] WDATA      [];
21   logic      WVALID;
22   logic      WLAST;
23   logic      WREADY;
24
25   axi_resp_e      BRESP;
26   logic      BVALID;
27   logic      BREADY;
28

```

```

////////////////////////////////////////////////
// 2. READ SIGNALS
////////////////////////////////////////////////

rand logic [ADDR_SIZE-1:0] ARADDR;
rand logic [      2:0] ARSIZE;
rand logic [      7:0] ARLEN;
logic      ARVALID;
logic      ARREADY;

logic      [DATA_SIZE-1:0] RDATA      [];
axi_resp_e      RRESP;
logic      RVALID;
logic      RLAST;
logic      RREADY;

////////////////////////////////////////////////
// 3. INTERNAL VARIABLES & HELPERS

```

```

class axi_transaction extends uvm_sequence_item;
    logic      [DATA_SIZE-1:0] sampled_rdata;
    int        last_addr;

    `uvm_object_utils_begin(axi_transaction)

        `uvm_field_array_int(WDATA, UVM_DEFAULT)
        `uvm_field_array_int(RDATA, UVM_DEFAULT)

        `uvm_field_int(AWADDR, UVM_DEFAULT)
        `uvm_field_int(ARADDR, UVM_DEFAULT)
        `uvm_field_enum(axi_resp_e, RRESP, UVM_DEFAULT)

```

```

class axi_transaction extends avm_sequence_item;

    constraint awaddr_c {
        AWADDR dist {
            16'd0           := 5,
            16'd4092        := 5,
            [16'd1 : 16'd4095] :/ 90
        };
    }

    ////////////////////////////////////////////
    // 5. READ CONSTRAINTS
    ////////////////////////////////////////////

    function new(string name = "axi_transaction");
        super.new(name);
    endfunction
endclass

`endif

}

```

axi_sequence.sv > Verilog-HDL/SystemVerilog/Bluespec SystemVerilog > AXI_SEQUENCE_SVH

```
1  `ifndef AXI_SEQUENCE_SVH
2  `define AXI_SEQUENCE_SVH
3
4
5  `include "axi_transaction.sv"
6  `include "uvm_macros.svh"
7  import uvm_pkg::*;
8  import param_pkg::*;
9
10 class axi_sequence extends uvm_sequence #(axi_transaction);
11     `uvm_object_utils(axi_sequence)
12
13     function new(string name = "axi_sequence");
14         super.new(name);
15     endfunction
```

```
    task send_directed(state_e dir, int len, int addr);
        axi_transaction req = axi_transaction::type_id::create("req");
        start_item(req);
        if (!req.randomize() with {
            write_read_e == dir;
            if (dir == WRITE) {
                AWLEN == len;
                AWADDR == addr;
            }
            if (dir == READ) {
                ARLEN == len;
                ARADDR == addr;
            }
        }) begin
            `uvm_fatal(get_type_name(), $sformatf("Failed to randomize %s. Len: %0d, Addr: %0d",
                dir.name(), len, addr))
        end
        finish_item(req);
    endtask
```

```
class axi_sequence extends uvm_sequence #(axi_transaction);
    task body();
        axi_transaction req, rd_req;
        repeat (100) begin
            req = axi_transaction::type_id::create("req");
            start_item(req);
            if (!req.randomize()) `uvm_fatal(get_type_name(), "Failed to randomize base traffic")
            finish_item(req);

            if (req.write_read_e == WRITE) begin
                rd_req = axi_transaction::type_id::create("rd_req");
                start_item(rd_req);
                rd_req.write_read_e = READ;
```

```

repeat (5) begin
    req = axi_transaction::type_id::create("req");
    start_item(req);
    if (!req.randomize()) with {
        ARLEN dist {
            8'd255 := 50,
            8'd0   := 50
        };
        write_read_e == READ;
    })
        `uvm_fatal(get_type_name(), "Failed read boundary dist")
    finish_item(req);
end

```

```

= axi_sequencer.sv > verible > % axi_sequencer

```

```

1  `ifndef AXI_SEQUENCER_SVH
2  `define AXI_SEQUENCER_SVH
3
4  `include "axi_transaction.sv"
5
6  `include "uvm_macros.svh"
7  import uvm_pkg::*;
8
9  class axi_sequencer extends uvm_sequencer #(axi_transaction);
10
11  `uvm_component_utils(axi_sequencer)

```

```

1  `ifndef AXI_DRIVER_SVH
2  `define AXI_DRIVER_SVH
3
4
5  `include "axi_transaction.sv"
6  `include "common_cfg.sv"
7  `include "uvm_macros.svh"
8  import uvm_pkg::*;
9  import param_pkg::*;
10
11  class axi_driver extends uvm_driver #(axi_transaction);
12
13      virtual axi_if.master master;
14      `uvm_component_utils(axi_driver)
15
16      function new(string name = "axi_driver", uvm_component parent);
17      |   super.new(name, parent);
18      endfunction
19
20      function void build_phase(uvm_phase phase);
21      |   super.build_phase(phase);
22
23      task run_phase(uvm_phase phase);
24      |   wait (master.ARESETn === 1'b1);
25      |   `uvm_info(get_type_name(), "Reset deasserted. Starting stimulus", UVM_LOW)
26
27      |   // Explicitly initialize signals to avoid X states
28      |   master.AWVALID = 0;
29      |   master.WVALID  = 0;
30      |   master.BREADY  = 0;
31      |   master.ARVALID = 0;
32      |   master.RREADY  = 0;

```

axi_driver.sv > Verilog-HDL/SystemVerilog/Bluespec SystemVerilog > axi_driver > drive_read

```
task axi_driver::drive_Write(axi_transaction req);
    @(negedge master.ACLK);
    master.AWADDR = req.AWADDR;
    master.AWSIZE = req.AWSIZE;
    master.AWLEN = req.AWLEN;
    master.AWVALID = 1'b1;

    while (master.AWREADY != 1'b1) @(negedge master.ACLK);
    @(negedge master.ACLK); // Hold for 1 full cycle, then clear
    master.AWVALID = 1'b0;
    `uvm_info(get_type_name(), "AW handshake done", UVM_HIGH)

    // W CHANNEL
    master.WVALID = 1'b1;
    for (int i = 0; i <= req.AWLEN; i++) begin
        master.WDATA = req.WDATA[i];
        master.WLAST = (i == req.AWLEN);

        while (master.WREADY != 1'b1) @(negedge master.ACLK);
        @(negedge master.ACLK); // Advance to next beat
    end
end
```

```
task axi_driver::drive_read(axi_transaction req);
    // AR CHANNEL
    @(negedge master.ACLK);
    master.ARADDR = req.ARADDR;
    master.ARSIZE = req.ARSIZE;
    master.ARLLEN = req.ARLLEN;
    master.ARVALID = 1'b1;

    while (master.ARREADY != 1'b1) @(negedge master.ACLK);
    @(negedge master.ACLK);
```

axi_monitor.sv > verible > %\$ axi_monitor

```
`ifndef AXI_MONITOR_SVH
`define AXI_MONITOR_SVH
```

```
`include "axi_transaction.sv"
```

```
`include "uvm_macros.svh"
```

```
import uvm_pkg::*;
```

```
class axi_monitor extends uvm_monitor;
```

```
    int W_beats, R_beats;
```

```
    virtual axi_if axi_vif;
```

```
    axi_transaction w_tr;
```

```
    axi_transaction r_tr;
```

```
class axi_monitor extends uvm_monitor;
```

```
function void build_phase(uvm_phase phase);
```

```
    super.build_phase(phase);
```

```
    `uvm_info(get_type_name(), "starting Monitor building phase", UVM_LOW)
```

```
    if (!uvm_config_db#(virtual axi_if)::get(this, "", "vif", axi_vif))
```

```
        `uvm_fatal(get_type_name(), "Failed to get the virtual interface")
```

```
endfunction
```



```
////////// WRITE MONITOR //////////
```

```
task write_tr();
    forever begin
        w_tr = axi_transaction::type_id::create("w_tr");

        @(posedge axi_vif.ACLK iff (axi_vif.AWVALID && axi_vif.AWREADY));
        w_tr.AWADDR = axi_vif.AWADDR;
        w_tr.AWSIZE = axi_vif.AWSIZE;
        w_tr.AWLEN = axi_vif.AWLEN;
        w_tr.WDATA = new[w_tr.AWLEN + 1];

        // WRITE CHANNEL

        for (int i = 0; i < (w_tr.AWLEN + 1); i++) begin
            @(posedge axi_vif.ACLK iff (axi_vif.WVALID && axi_vif.WREADY));
            w_tr.WDATA[i] = axi_vif.WDATA;
        end

        @(posedge axi_vif.ACLK iff (axi_vif.BVALID && axi_vif.BREADY));
        w_tr.BRESP = axi_resp_e'(axi_vif.BRESP);
        w_tr.write_read_e = WRITE;
```

```
////////// READ MONITOR //////////
```

```
task read_tr();
    forever begin
        r_tr = axi_transaction::type_id::create("r_tr");

        // ADDRESS READ CHANNEL

        @(posedge axi_vif.ACLK iff (axi_vif.ARVALID && axi_vif.ARREADY));
        r_tr.ARADDR = axi_vif.ARADDR;
```



```

`ifndef AXI_AGENT_SVH
`define AXI_AGENT_SVH

`include "axi_sequencer.sv"
`include "axi_monitor.sv"
`include "axi_driver.sv"

`include "uvm_macros.svh"
import uvm_pkg::*;

class axi_agent extends uvm_agent;

    axi_sequencer sqr;
    axi_driver drv;
    axi_monitor mon;
    uvm_active_passive_enum is_active = UVM_ACTIVE; // declare the active_passive enum

    `uvm_component_utils(axi_agent)

function new(string name = "axi_agent", uvm_component parent);
    super.new(name, parent);
endfunction

```

```

function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    `uvm_info(get_type_name(), "starting Agent building phase", UVM_LOW)

    if (!uvm_config_db#(uvm_active_passive_enum)::get(this, "", "is_active", is_active))
        `uvm_fatal(get_type_name(), "Failed to get the active state of agent")

    mon = axi_monitor::type_id::create("mon", this);
    if (is_active == UVM_ACTIVE) begin
        sqr = axi_sequencer::type_id::create("sqr", this);
        drv = axi_driver::type_id::create("drv", this);
    end
endfunction

function void connect_phase(uvm_phase phase);
    super.connect_phase(phase);
    if (is_active == UVM_ACTIVE) begin
        drv.seq_item_port.connect(sqr.seq_item_export);
    end
endfunction

endclass

`endif

```

axi_coverage.sv > verilog-HDL/SystemVerilog/bluespec SystemVerilog > [?] AXI_COVERAGE_SVH

```
1  `ifndef AXI_COVERAGE_SVH
2  `define AXI_COVERAGE_SVH
3
4
5  `include "axi_transaction.sv"
6
7
8  `include "uvm_macros.svh"
9  import uvm_pkg::*;
10
11  class axi_coverage extends uvm_component;
12
13      axi_transaction tr;
14      uvm_analysis_export #(axi_transaction) ex_write;
15      uvm_analysis_export #(axi_transaction) ex_read;
16      uvm_tlm_analysis_fifo #(axi_transaction) w_fifo;
17      uvm_tlm_analysis_fifo #(axi_transaction) r_fifo;
18      `uvm_component_utils(axi_coverage)
19
20      //////////////////////////////////////////
21      // 6. COVERGROUP
22      //////////////////////////////////////////
23  
```

```
covergroup w_axi_cg with function sample (axi_transaction tr);
    option.auto_bin_max = 0;
    option.per_instance = 1; // Highly recommended for class-based covergroups

    // =====
    // WRITE PHASE COVERAGE
    // =====

    // -- Write Protocol Vectors --
    W_address_regions: coverpoint tr.AWADDR {
        bins w_addr_min = {16'd0};
        bins w_addr_low = {[16'd1 : 16'd1000]};
        bins w_addr_mid = {[16'd1001 : 16'd3000]};
        bins w_addr_high = {[16'd3001 : 16'd4091]};
        bins w_addr_max = {16'd4092};
    }

    W_burst_length: coverpoint tr.AWSIZE {
        bins w_size = {3'd2}; illegal_bins illegal_w = {[3'd0 : 3'd1], [3'd3 : 3'd7]};
    }

    w_len: coverpoint tr.AWLEN {
        bins w_len_min = {0};
        bins w_len_low = {[8'd1 : 8'd80]};
        bins w_len_mid = {[8'd81 : 8'd160]};
        bins w_len_high = {[8'd161 : 8'd254]};
        bins w_len_max = {8'd255};
    }
}

```

```

w_resp: coverpoint tr.BRESP {bins okay = {OKAY}; bins slverr = {SLVERR};}

write_boundary: cross w_len, W_address_regions, w_resp{

    option.cross_auto_bin_max = 0;

    illegal_bins max = binsof(w_len.w_len_max) && binsof(W_address_regions.w_addr_max)
    && binsof(w_resp.slverr);

    bins mid_ = binsof(w_len.w_len_mid) && binsof(W_address_regions.w_addr_min)
    && binsof(w_resp.okay);

    bins min_ = binsof(w_len.w_len_min) && binsof(W_address_regions.w_addr_min)
    && binsof(w_resp.okay);
}
endgroup

covergroup r_axi_cg with function sample (axi_transaction tr);
    option.auto_bin_max = 0;
    option.per_instance = 1; // Highly recommended for class-based covergroups

    // =====
    // READ PHASE COVERAGE
    // =====

```

```

R_address_regions: coverpoint tr.ARADDR {
    bins r_addr_min = {16'd0};
    bins r_addr_low = {[16'd1 : 16'd1000]};
    bins r_addr_mid = {[16'd1001 : 16'd3000]};
    bins r_addr_high = {[16'd3001 : 16'd4091]};
    bins r_addr_max = {16'd4092};
}

R_burst_length: coverpoint tr.ARSIZE {
    bins r_size = {3'd2}; illegal_bins illegal_r = {[3'd0 : 3'd1], [3'd3 : 3'd7]};
}

r_len: coverpoint tr.ARLen {
    bins r_len_min = {0};
    bins r_len_low = {[8'd1 : 8'd80]};
    bins r_len_mid = {[8'd81 : 8'd160]};
    bins r_len_high = {[8'd161 : 8'd254]};
    bins r_len_max = {8'd255};
}

r_resp: coverpoint tr.RRESP {bins okay = {OKAY}; bins slverr = {SLVERR};}

read_boundary: cross r_len, R_address_regions, r_resp{

    option.cross_auto_bin_max = 0;

    bins max = binsof(r_len.r_len_max) && binsof(R_address_regions.r_addr_max)
    && binsof(r_resp.slverr);

    bins mid_ = binsof(r_len.r_len_mid) && binsof(R_address_regions.r_addr_min)
    && binsof(r_resp.okay);
}

```

```

        bins_min_ = binsof(r_len.r_len_min) && binsof(R_address_regions.r_addr_min)
        && binsof(r_resp.okay);
    }
endgroup

function new(string name = "axi_coverage", uvm_component parent);
    super.new(name, parent);
    w_axi_cg = new();
    r_axi_cg = new();
endfunction

function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    `uvm_info(get_type_name(), "starting Coverage building", UVM_LOW);

    ex_write = new("ex_write", this);
    ex_read = new("ex_read", this);
    w_fifo = new("w_fifo", this);
    r_fifo = new("r_fifo", this);
endfunction

function void connect_phase(uvm_phase phase);
    `uvm_info(get_type_name(), "starting Coverage connection", UVM_LOW);
    ex_write.connect(w_fifo.analysis_export);
    ex_read.connect(r_fifo.analysis_export);
endfunction

```

```

    task run_phase(uvm_phase phase);
        `uvm_info(get_type_name(), "starting Coverage run", UVM_LOW);
        fork
            begin
                forever begin
                    axi_transaction w_tr;
                    w_fifo.get(w_tr);
                    w_axi_cg.sample(w_tr);
                end
            end

            begin
                forever begin
                    axi_transaction r_tr;
                    r_fifo.get(r_tr);
                    r_axi_cg.sample(r_tr);
                end
            end
        join
    endtask
endclass

`endif

```

```

`ifndef AXI_SCOREBOARD_SVH
`define AXI_SCOREBOARD_SVH

`include "axi_transaction.sv"
`include "common_cfg.sv"

`include "uvm_macros.svh"
import uvm_pkg::*;
import param_pkg::*;

class axi_scoreboard extends uvm_scoreboard;

    semaphore golden_sem;
    axi_transaction tr;
    logic [DATA_SIZE-1:0] golden_arr[logic [ADDR_SIZE-1:0]];
    uvm_analysis_export #(axi_transaction) ex_write;
    uvm_analysis_export #(axi_transaction) ex_read;
    uvm_tlm_analysis_fifo #(axi_transaction) w_fifo;
    uvm_tlm_analysis_fifo #(axi_transaction) r_fifo;
    `uvm_component_utils(axi_scoreboard)

    covergroup axi_cg;

    ////////////coverage Logic////////////////////////////////////

endgroup

function new(string name = "axi_scoreboard", uvm_component parent);
    super.new(name, parent);
    axi_cg = new();
endfunction

```

```

function new(string name = "axi_scoreboard", uvm_component parent);
    axi_cg = new();
endfunction

function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    `uvm_info(get_type_name(), "starting Coverage building", UVM_LOW);

    ex_write = new("ex_write", this);
    ex_read  = new("ex_read", this);
    w_fifo   = new("w_fifo", this);
    r_fifo   = new("r_fifo", this);
endfunction

function void connect_phase(uvm_phase phase);
    `uvm_info(get_type_name(), "starting Coverage connection", UVM_LOW);
    ex_write.connect(w_fifo.analysis_export);
    ex_read.connect(r_fifo.analysis_export);
endfunction

```

```

task run_phase(uvm_phase phase);
    axi_transaction r_tr;
    axi_transaction w_tr;
    `uvm_info(get_type_name(), "starting Coverage run", UVM_LOW);
    fork
        forever begin
            w_fifo.get(w_tr);
            golden_model(w_tr);
        end

        forever begin
            r_fifo.get(r_tr);
            check_data(r_tr);
        end
    join
endtask

extern task golden_model(axi_transaction tr);
extern task check_data(axi_transaction tr);

endclass

```



```

task axi_scoreboard::golden_model(axi_transaction tr);
    int unsigned address = int'(tr.AWADDR >> 2);

    if (tr.BRESP == 2'b10) begin
        `uvm_info("SCB_OOB_WRITE_DETECTED", $sformatf(
            "DUT correctly rejected out-of-bounds write at byte addr %h (word addr %h) with SLVERR",
            tr.AWADDR, address), UVM_HIGH)
        return;
    end

    for (int i = 0; i < (tr.AWLEN + 1); i++) begin
        if (address < 1024) begin
            golden_arr[address] = tr.WDATA[i];
        end else begin
            `uvm_info("SCB_OOB_WRITE_DETECTED", $sformatf(
                "DUT correctly rejected write at out-of-bounds word addr %h", address), UVM_HIGH)
            end
            address++;
        end
    end
endtask

```

```

axi_scoreboard.sv > verible > [E] address
task axi_scoreboard::check_data(axi_transaction tr);
    int unsigned address;
    address = int'(tr.ARADDR >> 2);

    if (tr.write_read_e != READ) begin
        `uvm_error("SCB_WRONG_TR", "check_data received a non-READ transaction!")
        return;
    end

    for (int i = 0; i < (tr.ARLen + 1); i++) begin

        if (tr.RRESP == 2'b10) begin
            `uvm_info("SCB_OOB_DETECTED", $sformatf(
                "DUT correctly returned SLVERR at word addr %h (byte addr %h) - out-of-bounds or 4KB boundary violation",
                address, address << 2), UVM_HIGH)
            return;
        end else begin

            if (golden_arr.exists(address)) begin
                if (golden_arr[address] == tr.RDATA[i])
                    `uvm_info("SCB_PASS", $sformatf("MATCH! Addr: %h | Expected: %h | Actual: %h",
                        address, golden_arr[address], tr.RDATA[i]), UVM_HIGH)
                else
                    `uvm_error("SCB_MISMATCH", $sformatf("MISMATCH! Addr: %h | Expected: %h | Actual: %h",
                        address, golden_arr[address], tr.RDATA[i]))
            end
        end
    end
endtask

```

```

        end else begin
            if (tr.RDATA[i] == 32'h0000_0000)
                `uvm_info("SCB_PASS_UNINIT", $sformatf(
                    "Correctly read 0 from unwritten addr: %h", address), UVM_HIGH)
            else
                `uvm_error("SCB_FAIL_UNINIT", $sformatf(
                    "Unwritten addr %h returned %h instead of 0!", address, tr.RDATA[i]))
            end
        end
        address++;
    end
endtask

`endif

```

```

xi_env.sv > Verible > ❗ axi_env > 📖 build_phase
`ifndef AXI_ENV_SVH
`define AXI_ENV_SVH

`include "axi_agent.sv"
`include "axi_coverage.sv"
`include "axi_scoreboard.sv"

`include "uvm_macros.svh"
import uvm_pkg::*;

class axi_env extends uvm_env;

    axi_agent agt;
    axi_scoreboard scb;
    axi_coverage cov;
    `uvm_component_utils(axi_env)

    function new(string name = "axi_env", uvm_component parent);
    |   super.new(name, parent);
    endfunction

    function void build_phase(uvm_phase phase);
    |   super.build_phase(phase);
    |   `uvm_info(get_type_name(), "starting Environment building phase", UVM_LOW)

    |   agt = axi_agent::type_id::create("agt", this);
    |   cov = axi_coverage::type_id::create("cov", this);
    |   scb = axi_scoreboard::type_id::create("scb", this);
    endfunction

```

```

    function void connect_phase(uvm_phase phase);
    |   super.connect_phase(phase);
    |   agt.mon.ap_write.connect(scb.ex_write);
    |   agt.mon.ap_read.connect(scb.ex_read);
    |   agt.mon.ap_write.connect(cov.ex_write);
    |   agt.mon.ap_read.connect(cov.ex_read);
    endfunction

endclass

`endif

```

```

`ifndef AXI_TEST_SVH
`define AXI_TEST_SVH

`include "axi_sequence.sv"
`include "axi_env.sv"
`include "common_cfg.sv"

`include "uvm_macros.svh"
import uvm_pkg::*;

class axi_test extends uvm_test;

    common_cfg m_cfg;
    axi_sequence seq;
    axi_env env;
    `uvm_component_utils(axi_test)

    function new(string name = "axi_test", uvm_component parent);
        super.new(name, parent);
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        `uvm_info(get_type_name(), "starting Test building phase", UVM_LOW)

        m_cfg = common_cfg::type_id::create("m_cfg");
        seq = axi_sequence::type_id::create("seq");
        env = axi_env::type_id::create("env", this);
        uvm_config_db#(common_cfg)::set(this, "*", "m_cfg", m_cfg);
    endfunction

```

```

        function void end_of_elaboration_phase(uvm_phase phase);
            super.end_of_elaboration_phase(phase);
            uvm_top.print_topology();
        endfunction

        task run_phase(uvm_phase phase);

            phase.raise_objection(this);
            seq.start(env.agt.sqr);
            #10ns;
            phase.drop_objection(this);

        endtask

    endclass

`endif

```



```

TOP.sv > Verible > TOP > dut
`include "uvm_macros.svh"
import uvm_pkg::*;
import axi_pkg::*;
import param_pkg::*;

module TOP;

    bit clk = 0;
    bit rst_n = 0;

    initial begin
        forever begin
            #5ns clk = ~clk;
        end
    end

    initial begin
        rst_n = 0;
        #20ns rst_n = 1;
    end
end

```

```

axi_if axi_vif (
    .ACLK(clk),
    .ARESETn(rst_n)
);
my_axi4_memory memory (axi_vif.mem_ram);
my_axi4 #(
    .DATA_WIDTH  (DATA_SIZE),
    .ADDR_WIDTH  (ADDR_SIZE),
    .MEMORY_DEPTH(DEPTH_SIZE) // Use the values from param_pkg
) dut (
    .slave (axi_vif.slave),
    .memory(axi_vif.mem_ctrl)
);

initial begin
    uvm_config_db#(uvm_active_passive_enum)::set(null, "uvm_test_top.env.agt", "is_active",
                                                UVM_ACTIVE);
    uvm_config_db#(virtual axi_if)::set(null, "uvm_test_top.env.agt.*", "vif", axi_vif);
    uvm_config_db#(virtual axi_if.master)::set(null, "uvm_test_top.env.agt.*", "master_vif",
                                                axi_vif.master);
    run_test("axi_test");
end

endmodule

```


≡ run.do

```
1  vdel -all -lib work
2  vlib work
3
4  # 1. Instrument code for coverage during compilation
5  vlog -f files.txt +cover
6
7  # 2. Enable coverage collection during simulation
8  vsim -voptargs="+acc" -coverage work.TOP \
9      -l full_simulation.log \
10     +UVM_VERBOSITY=UVM_LOW
11
12  run -all
13
14  # 3. Save the binary coverage database
15  coverage save axi_coverage.ucdb
16
17  # 4. Generate the detailed text report
18  coverage report -file coverage_report.txt -detail -all
```

Covergroups								
Name	Class Type	Coverage	Goal	% of Goal	Status	Included	Merge_instances	Get_inst_coverage
[-] /axi_pkg/axi_coverage		100.00%						
[-] TYPE w_axi_cg		100.00%	100	100.00...	 ✓		auto(1)	
[-] TYPE r_axi_cg		100.00%	100	100.00...	 ✓		auto(1)	

```

# UVM_INFO: /questasim64_2021.1/win64/..//verilog_src/uvm-1.1d/src/base/uvm_root.svh(430)
# --- UVM Report Summary ---
#
# ** Report counts by severity
# UVM_INFO : 17
# UVM_WARNING : 0
# UVM_ERROR : 0
# UVM_FATAL : 0
# ** Report counts by id
# [RNTST] 1
# [TEST_DONE] 1
# [UVMTOP] 1
# [axi_agent] 1
# [axi_coverage] 3
# [axi_driver] 2
# [axi_env] 1
# [axi_monitor] 2
# [axi_scoreboard] 3
# [axi_sequencer] 1
# [axi_test] 1
# ** Note: $finish : C:/questasim64_2021.1/win64/..//verilog_src/uvm-1.1d/src/base/uvm_root.svh(430)
# Time: 508850 ns Iteration: 54 Instance: /TOP
# 1

```